



Towards Dev · [Follow publication](#)

C++20 `std::latch` and `std::barrier`: Mechanical Sympathy in Concurrency

Move beyond basic mutexes and semaphores to build highly efficient, phase-based synchronization in modern C++ systems.

9 min read · May 19, 2026



Sagar

Following ▾

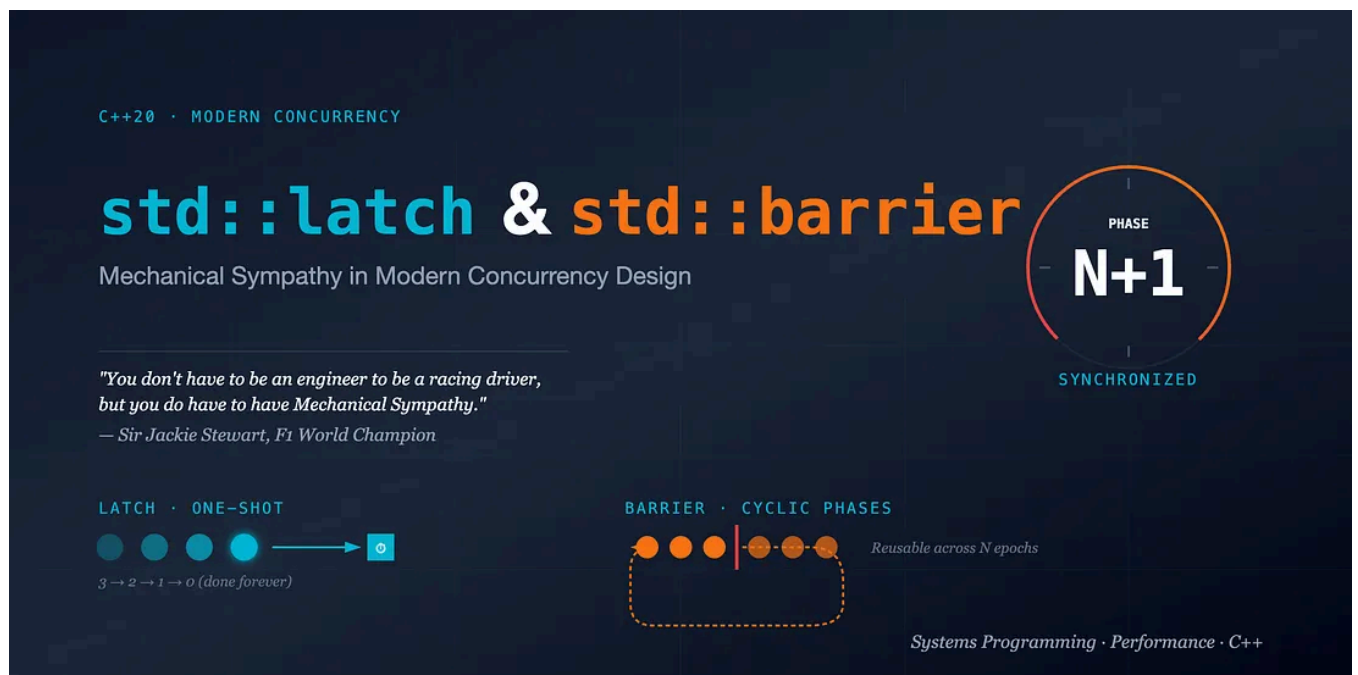


Listen



Share

⋮ More



“You don’t have to be an engineer to be a racing driver, but you do have to have Mechanical Sympathy.”

— Sir Jackie Stewart, three-time Formula 1 World Champion

In the world of C++ systems programming, the same idea holds:

*“You don’t have to be a concurrency expert to write multithreaded code, but you do have to have **Mechanical Sympathy** — an intuitive sense of how the hardware actually behaves under concurrency.”*

In modern C++ systems, the real challenge of multithreading isn’t just protecting shared data — it’s coordinating *when* work happens.

We’ve long used `std::mutex`, `std::condition_variable`, and, since C++20, `std::counting_semaphore` and `std::binary_semaphore` to express these relationships. These are high-level, well-designed abstractions that don’t require hand-rolled synchronization.

Still, they’re generally **multi-purpose mechanisms**: you encode specific coordination patterns indirectly through counters, signals, and careful ordering. C++20 introduces `std::latch` and `std::barrier` to model coordination patterns **explicitly**, so the intent is clear from the type itself. In the next sections, we’ll see the specific synchronization patterns each one is designed for and how they simplify real-world code.

`std::latch` : **Downward Counters and Lock-Free Arrival Checkpoints**

Imagine we are spinning up an inference server for a massive neural network (like a multi-gigabyte LLM). Before the server can accept requests, several initialization tasks must complete:

- Partition the model weights into multiple shards
- Load each shard in parallel across worker threads
- Maximize disk I/O and memory bandwidth during startup
- Wait until every shard is fully loaded before opening the server socket

This is exactly the kind of one-time coordination problem `std::latch` is designed for.

A `std::latch` is a single-use countdown counter. Threads decrement the counter, and waiting threads block until the counter hits zero.

So let's write some code for this example:

```
#include <iostream>
#include <vector>
#include <thread>
#include <latch>
#include <chrono>

// Mock function representing disk I/O and memory mapping
void load_model_shard(int shard_id, std::latch& sync_latch) {
    std::cout << "Thread " << shard_id << ": Loading weight shard...\n";
    std::this_thread::sleep_for(std::chrono::milliseconds(100 * shard_id));

    std::cout << "Thread " << shard_id << ": Shard loaded.\n";

    // Atomically decrement the latch.
    // This releases waiters but DOES NOT block this worker thread.
    sync_latch.count_down();

    // Demonstrate independent post-latch execution
    std::cout << "Thread " << shard_id << ": Releasing temp buffers and exiting\n";
}

int main() {
    constexpr int NUM_SHARDS = 3;

    // Initialize the latch with the exact number of tasks
    std::latch weights_latch(NUM_SHARDS);
    std::vector<std::jthread> workers;

    std::cout << "[Main] Kicking off parallel shard loading...\n";

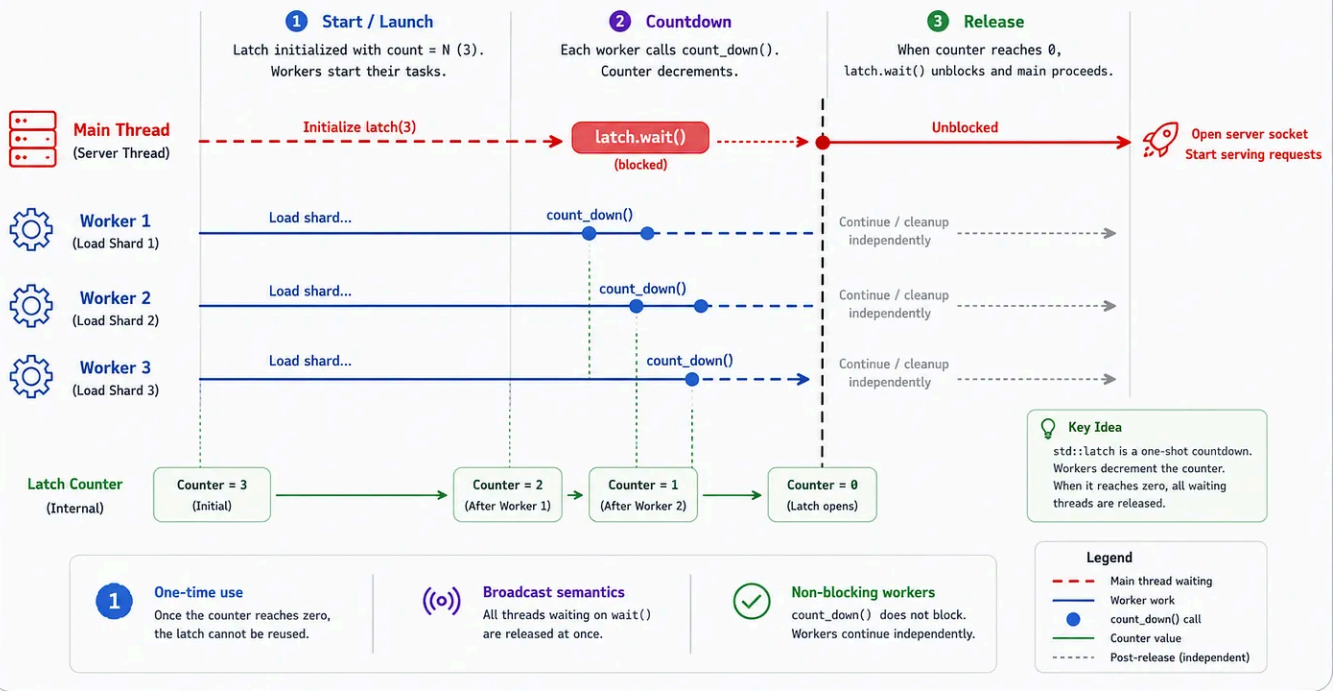
    for (int i = 1; i <= NUM_SHARDS; ++i) {
        // C++20 jthread automatically joins on destruction
        workers.emplace_back(load_model_shard, i, std::ref(weights_latch));
    }

    // Main thread sleeps efficiently until the counter reaches exactly 0
    weights_latch.wait();

    std::cout << "[Main] All weights loaded. Starting inference server!\n";
    return 0;
}
```

std::latch – One-time Countdown Synchronization

The main thread waits until the latch counter reaches zero.



When `std::latch`'s counter reaches zero:

1. All threads currently blocked on `wait()` are immediately unblocked and proceed.
2. Any future calls to `wait()` or `try_wait()` return immediately without blocking, because the latch is already “done.”
3. The latch is single-use: the counter cannot be incremented or reset after reaching zero. Any further `count_down()` calls are undefined behavior (the counter is already at its minimum)

Why not `std::counting_semaphore`?

A common question might come to your mind: “*Can't I just use `std::counting_semaphore` for this?*”

Mechanically, yes. You could initialize a semaphore to 0, have the workers release, and have the main thread loop and acquire N times. But semantically and architecturally, this is a flawed choice.

1. **Ownership Semantics:** Semaphores are designed to model *resource pools* (e.g., connection pools, bounded queues). Latches are designed purely for *thread coordination*. Using a semaphore to coordinate state obscures your intent.
2. **Broadcast Capability:** `std::latch` natively supports multiple threads waiting simultaneously. When the latch hits zero, *all* waiting threads are unblocked. With a semaphore, one thread calling `acquire()` consumes the signal, keeping others blocked.
3. **Synchronization Overhead:** Recreating phased coordination with semaphores often requires additional bookkeeping and can introduce unnecessary synchronization overhead, rather than utilizing a primitive natively optimized for rendezvous.

Lets build upon our example to understand `std::barrier` .

Upgrading the Loop: The Case for `std::barrier`

The ML model is loaded, but now we want to actually *train* it.

Training is an iterative process divided into Epochs. In a distributed training loop, multiple threads calculate gradients on their local data batches. At the end of every epoch, they must all pause, perform a reduction step (conceptually similar to an *all-reduce* across threads), update the global model, and then seamlessly begin the next epoch.

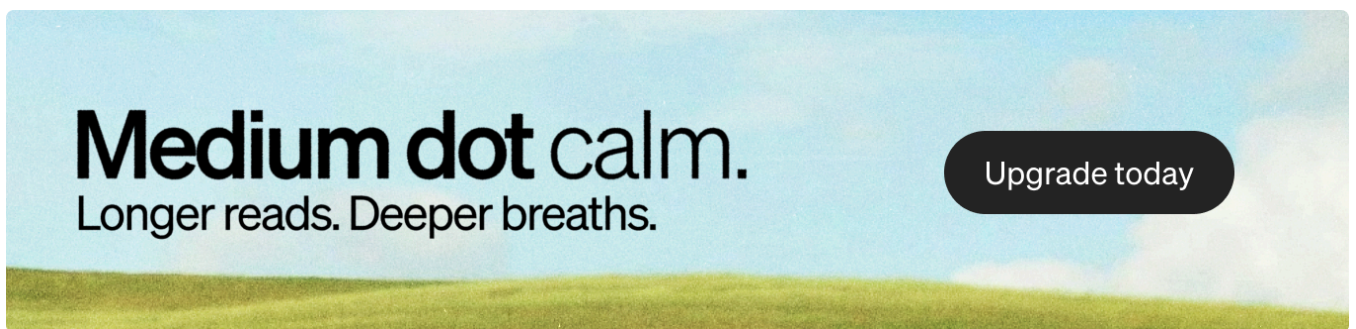
A `std::latch` won't work here because **latches are strictly single-use**.

- Once the internal counter reaches exactly 0, the **latch** is permanently spent; it cannot be reset or reused.
- If you want another synchronization point, you must instantiate a completely new `std::latch` .
- Since these primitives are typically passed by reference to workers, this creates a severe architectural disadvantage in a long-lived **threadpool** model— you would either have to tear down and recreate your threads, or introduce messy pointer-swapping logic to pass the new latch.

Choosing the right synchronization primitive requires a clear understanding of your workload's lifecycle. To specifically accommodate the reusability required for **phase-based coordination patterns** without destroying your threads, C++20 introduces `std::barrier`.

A barrier is reusable. It operates in phases. Even better, `std::barrier` allows you to inject a completion function: a callable that executes exactly once when all threads hit the barrier, right before the next phase begins.

Without `std::barrier`, you'd typically need a dedicated coordinator thread or an extra synchronization round. The completion function eliminates that entire class of architectural overhead.



Architectural Caveat: Because the completion phase executes serially (in the context of the last thread to arrive), expensive work inside the callback can become a scalability bottleneck. Keep it light.

```
#include <iostream>
#include <vector>
#include <thread>
#include <barrier>
#include <syncstream>

int main() {
    constexpr int NUM_THREADS = 3;
    constexpr int NUM_EPOCHS = 2;

    // The completion function is required to be noexcept in C++20.
    // It runs serially in the context of the thread that arrives last.
    auto on_epoch_done = []() noexcept {
        // Use osyncstream to prevent interleaved I/O from multiple threads
        std::osyncstream(std::cout)
            << "\n--- [Barrier Phase] All-Reduce Gradients & Update Model ---\n";
    };

    // Initialize barrier with thread count and the completion callback
```

```

std::barrier sync_point(NUM_THREADS, on_epoch_done);

auto training_loop = [&](int thread_id) {
    for (int epoch = 1; epoch <= NUM_EPOCHS; ++epoch) {
        std::osyncstream(std::cout)
            << "Thread " << thread_id << ": Calculating gradients for Epoch

        // Simulate heavy ML work
        std::this_thread::sleep_for(std::chrono::milliseconds(100));

        // Arrive at the barrier and wait for all peers.
        // The last thread to arrive triggers on_epoch_done() automatically
        // Once the callback finishes, the barrier resets and threads are r
        sync_point.arrive_and_wait();

        std::osyncstream(std::cout)
            << "Thread " << thread_id << ": Starting next phase.\n";
    }
};

std::vector<std::jthread> trainers;
for (int i = 1; i <= NUM_THREADS; ++i) {
    trainers.emplace_back(training_loop, i);
}

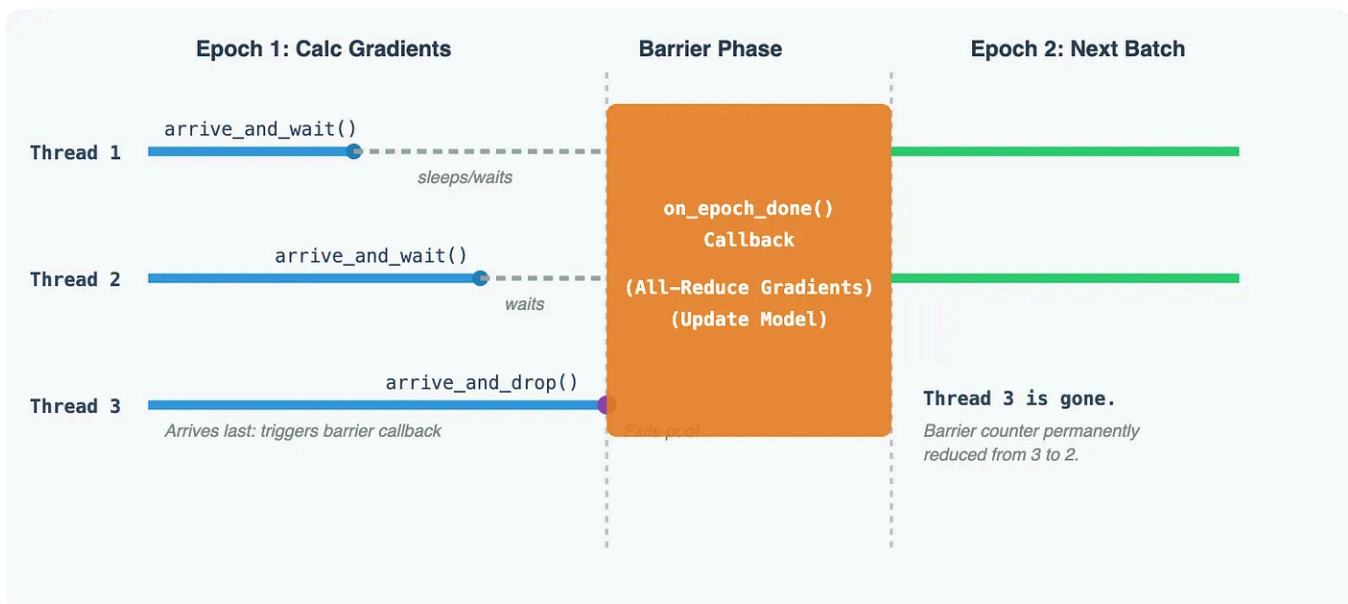
return 0;
}

```

arrive_and_wait() : Tells the barrier, *"I am done with this phase, and I will be back for the next one."* If all 3 threads call this, the barrier resets its expected count back to 3 for the next epoch.

See the visual diagram:





Dropping Threads Dynamically

You might notice `arrive_and_drop()` in the diagram above. `std::barrier` allows threads to permanently exit the synchronization loop using this method.

`arrive_and_drop()` : Tells the barrier, *"I am done with this phase, and I am leaving the thread pool permanently."* This decrements the expected count for the current phase, AND it permanently decrements the expected count for all subsequent phases.

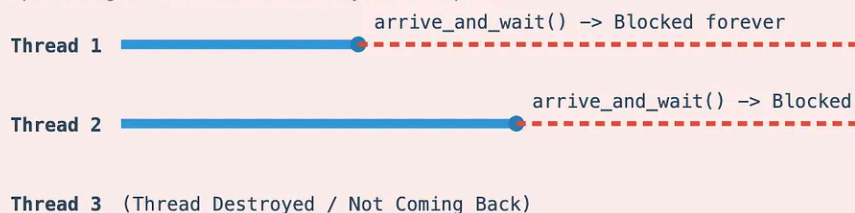
This is critical in real systems where worker threads may exit early (e.g., uneven data sharding or dynamically scaling down pool size), preventing deadlocks in long-running phases where remaining threads would otherwise wait forever for a dropped peer.

Why This Design and Why Without the `arrive_and_drop()` Deadlock happens:

Imagine if it *did* reset to 3 after Thread 3 dropped. In Epoch 2, Thread 1 and Thread 2 would do their work, call `arrive_and_wait()`, and then... **deadlock**. They would sit there sleeping forever, waiting for a 3rd thread that has exited and is never coming back.

Scenario A (The Problem): What if the barrier reset back to 3?

Epoch 2 begins, but Thread 3 has already exited the pool.

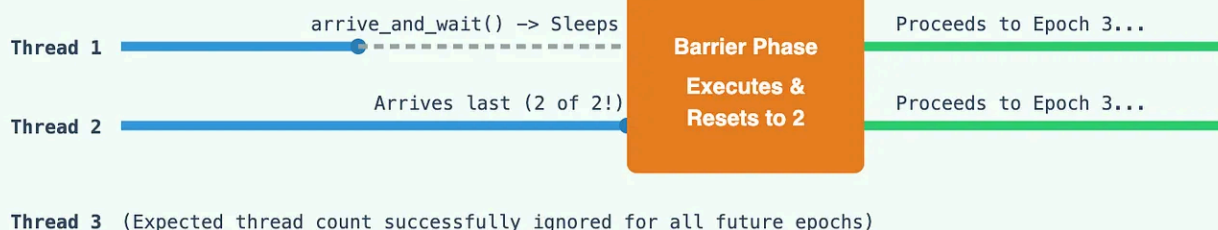


DEADLOCK

Threads 1 & 2 wait forever for a 3rd arrival that will never happen.

Scenario B (C++20 Solution): Thread 3 used `arrive_and_drop()`

Barrier permanently reduced its requirement to exactly 2 threads.



By automatically reducing the expected count to 2, `std::barrier` allows the surviving threads to continue cycling through future epochs without deadlocking.

This is exactly why `std::barrier` is superior to a manual `std::condition_variable` counter for dynamic thread pools. Handling dropping threads manually requires extremely careful locking to update the "total expected threads" variable without causing race conditions. C++20 handles it safely in a single atomic operation.

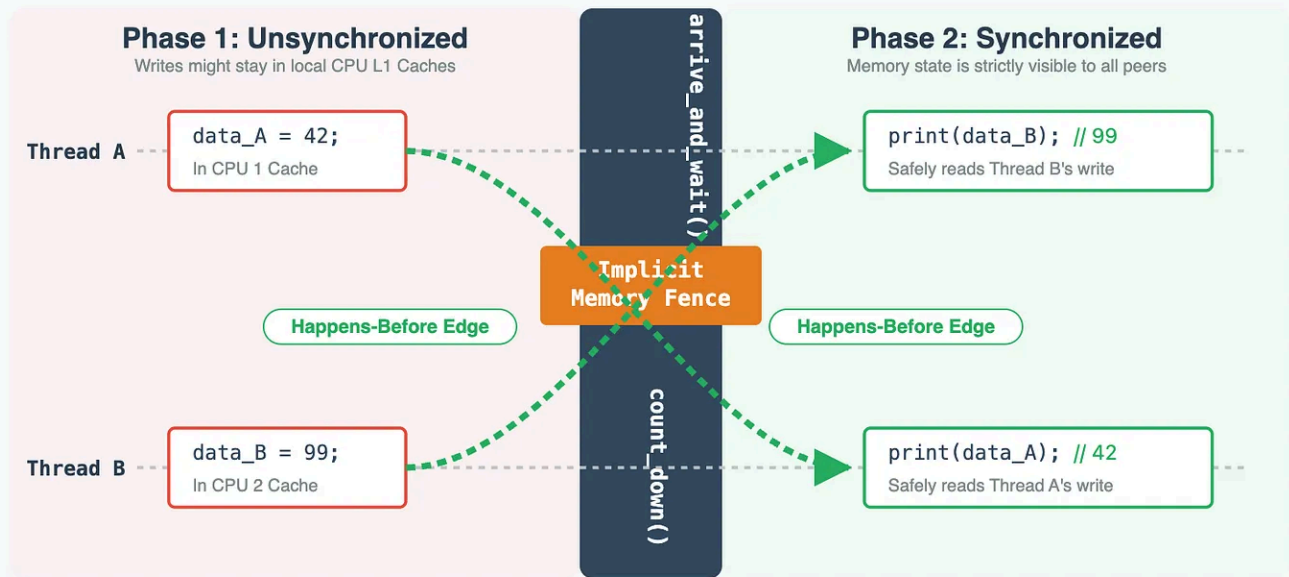
A Note on Memory Visibility

Beyond control flow, both `std::latch` and `std::barrier` establish critical synchronization points within the C++ memory model.

When thread A performs computations, those memory writes are not automatically visible to thread B due to CPU caching and compiler reordering. However, writes performed *before* a call to `count_down()` or `arrive_and_wait()` are strictly guaranteed to *happen-before* the unblocking of threads exiting the synchronization phase. You do not need explicit `std::atomic_thread_fence` calls; the primitives guarantee your memory state is visible across all peers before the next phase begins.

Memory Visibility & The Happens-Before Guarantee

How `std::barrier` and `std::latch` flush CPU caches without manual memory fences.



A Note on False Sharing

As a final systems-level detail, consider where your `std::latch` or `std::barrier` lives in memory.

These primitives contain highly contended atomic variables. If you place a `std::latch` in a `struct` right next to frequently updated worker data, you will induce **false sharing**.

The hardware prefetcher will pull the latch and the worker data into the same L1 cache line (64 bytes on most x86/ARM architectures). Every time a thread updates its local data, it will accidentally invalidate the cache line containing the latch for all other threads, destroying the performance of the synchronization point.

To prevent this, align your synchronization primitives to the hardware destructive interference size:

```
#include <new>    // for std::hardware_destructive_interference_size
#include <latch>

struct ServerState {
    // Force the latch to sit on its own cache line
```

```

alignas(std::hardware_destructive_interference_size) std::latch sync_point{

// Other state...
int active_connections;
};

```

Without alignas: False Sharing (Cache Thrashing)

Variables are packed tightly. CPU 2's data update invalidates CPU 1's latch synchronization.

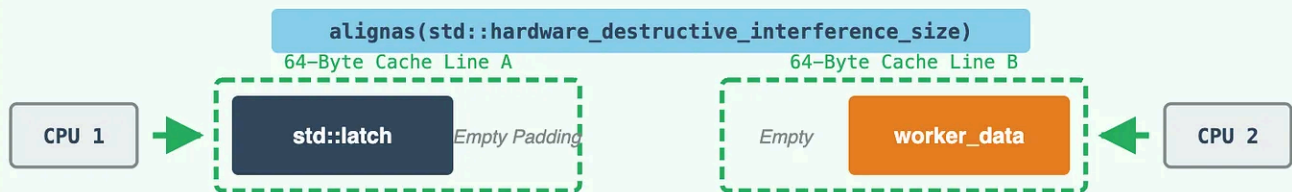
Single 64-Byte L1 Cache Line



Hardware destroys & re-fetches the ENTIRE cache line on every write!

With alignas: Mechanical Sympathy via Cache Isolation

Padding forces variables into completely separate cache lines. Updates no longer overlap.



✓ Zero Contention. Both cores run at maximum L1 hardware speed.

This ensures the OS and CPU can manage the rendezvous point efficiently without cache-coherency crossfire from unrelated data.

Under the Hood: Futexes and Phase Counters

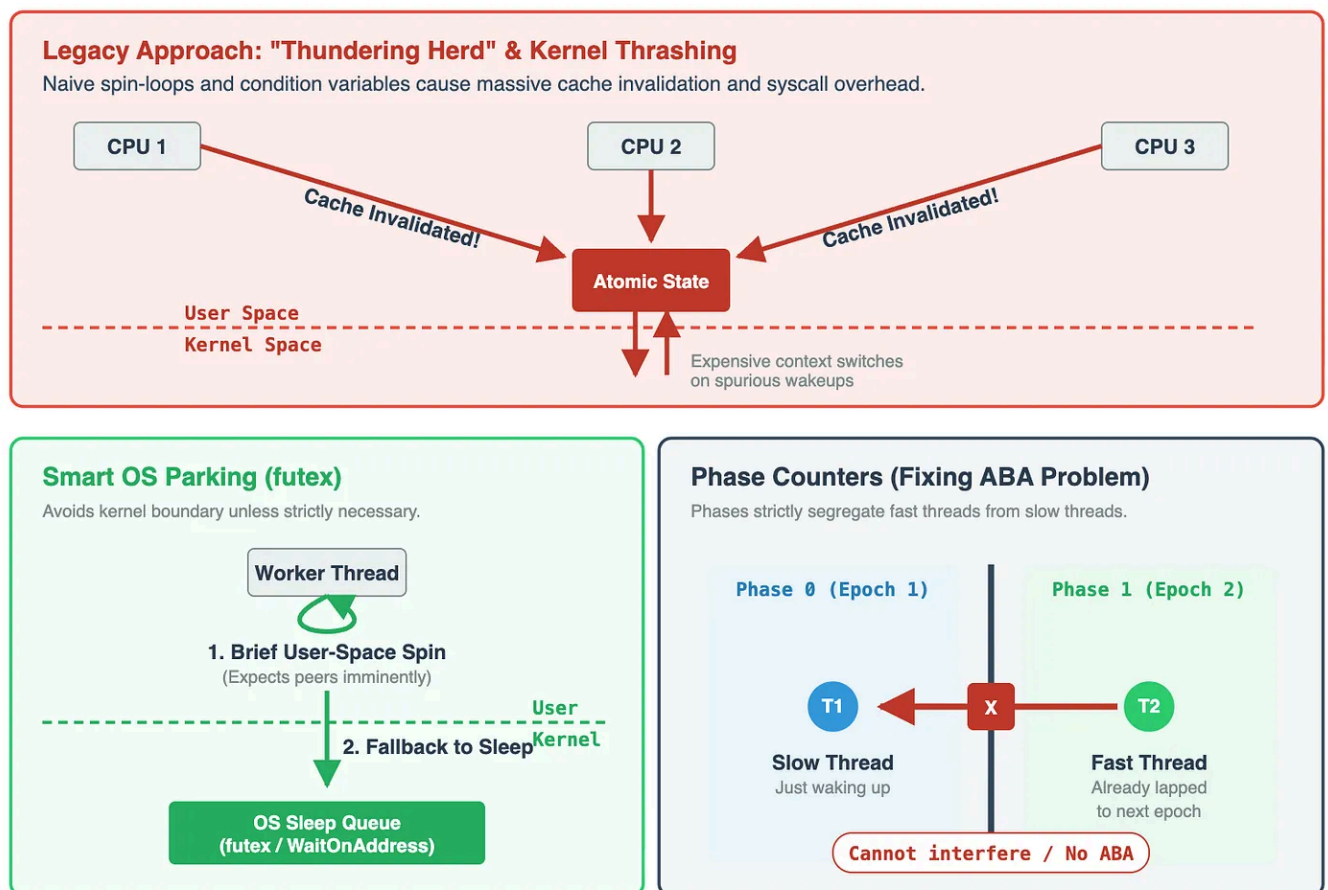
The real advantage of these primitives isn't syntax — it's how they map to OS and CPU behavior.

Naive `std::condition_variable` rendezvous often suffer from spurious wakeups and cache-line contention. When multiple threads spin on a shared atomic, they continuously invalidate each other's caches—the classic “thundering herd.”

Modern implementations of `std::latch` and `std::barrier` avoid this. They spin briefly in user space, then fall back to OS-level parking (e.g., futex on Linux,

WaitOnAddress on Windows), putting threads to sleep efficiently instead of burning CPU.

`std::barrier` also uses **phase counters** to separate synchronization rounds. Without this, a fast thread could “lap” a slow one (**the ABA problem**), entering the next phase while others are still waking up. Phase tracking ensures threads from different epochs never interfere.



Executive Summary: Latch vs Barrier

A good mental model is: use a latch when there is one finish line; use a barrier when there are many laps

Feature	<code>std::latch</code>	<code>std::barrier</code>
Lifecycle	Single-use.	Reusable (multi-phase).
Decrement	A thread can decrement by 1 or more.	Strictly decrements by 1 per arrival.
Completion Phase	None.	Custom noexcept callback executed automatically.
Thread Dropping	Not supported.	Threads can gracefully exit the loop using arrive_and_drop() .
Primary Use Case	Waiting for N asymmetric tasks to finish.	Coordinated cyclic execution among identical worker threads.

Closing Thoughts

Concurrency in modern C++ no longer requires hand-rolling fragile `condition_variable` loops or fighting the OS scheduler.

`std::latch` and `std::barrier` give us the exact tools we need to achieve true mechanical sympathy—minimizing cache thrashing, avoiding the ABA problem, and keeping threads out of the kernel whenever possible.

By reaching for these primitives, you aren't just writing less boilerplate, you are explicitly signaling your architectural intent to the compiler, the hardware, and the next engineer who reads your code. It's time to let the standard library do the heavy lifting.

*If you're serious about passing technical interviews, try **PracHub**. It helped me structure my preparation with advanced mock sessions. [[Check it out here](#)] and start practicing. (Disclosure: This is an affiliate link).*

Found this article helpful? Please **Clap** 🙌 and **follow** for more C++ and system programming content.